

Reactive Endocrine Substrate

```
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlin.math.min
import kotlin.math.max

data class HormoneProfile(
    [span_5](start_span)[span_6](start_span)// Mutable internal state for the Endocrine
    System[span_5](end_span)[span_6](end_span)
    internal val _cortisol = MutableStateFlow(0.3f),
    internal val _dopamine = MutableStateFlow(0.5f),
    internal val _oxytocin = MutableStateFlow(0.3f),

    [span_7](start_span)// Public immutable flows for the Brain to observe[span_7](end_span)
    val cortisol: StateFlow<Float> = _cortisol.asStateFlow(),
    val dopamine: StateFlow<Float> = _dopamine.asStateFlow(),
    val oxytocin: StateFlow<Float> = _oxytocin.asStateFlow()
)

[span_8](start_span)[span_9](start_span)class EndocrineSystem {
//[span_8](end_span)[span_9](end_span)
    [span_10](start_span)[span_11](start_span)val hormones = HormoneProfile()
//[span_10](end_span)[span_11](end_span)

    // Called asynchronously when external stimuli occur
    suspend fun processStressEvent(intensity: Float) {
        // Cortisol spikes immediately, affecting all observing coroutines
        val current = hormones._cortisol.value
        hormones._cortisol.emit(min(1.0f, current + (intensity * 0.5f)))
    }

    // Simulates Homeostasis over time
    suspend fun metabolizeHormones() {
        hormones._cortisol.emit(max(0.1f, hormones._cortisol.value - 0.01f))
        // ... metabolic decay for others
    }
}
```

2. Interrupt-Driven Spinal Cord

```
import kotlinx.coroutines.channels.Channel
import kotlinx.coroutines.coroutineScope
```

```
import kotlinx.coroutines.launch
```

```
[span_13](start_span)[span_14](start_span)class SpinalCord {  
//[span_13](end_span)[span_14](end_span)  
    // CONFLATED channel ensures we only care about the most recent, urgent reflex  
    val reflexInterruptChannel = Channel<SensorData>(Channel.CONFLATED)  
  
    suspend fun processSensoryInput(input: SensorData) {  
        [span_15](start_span)if (input.isPainful) { //[span_15](end_span)  
            [span_16](start_span)// Instantly bypass normal processing[span_16](end_span)  
            [span_17](start_span)triggerReflex("WITHDRAW") //[span_17](end_span)  
            // Fire interrupt to cancel/pause brain coroutines  
            reflexInterruptChannel.send(input)  
        } else {  
            // Route to standard brain processing  
        }  
    }  
  
    [span_18](start_span)private fun triggerReflex(action: String) { //[span_18](end_span)  
        println("REFLEX ENGAGED: $action. Bypassing cognition.")  
    }  
}
```

Gradient-Bounded Reasoning (The Brain)

```
class AndroidAIBrain(  
    [span_23](start_span)[span_24](start_span)private val memory: MemoryEngine,  
//[span_23](end_span)[span_24](end_span)  
    [span_25](start_span)private val endocrineSystem: EndocrineSystem //[span_25](end_span)  
) {  
    suspend fun process(input: SensorData): Decision = coroutineScope {  
        [span_26](start_span)[span_27](start_span)// 1. Read current chemical  
state[span_26](end_span)[span_27](end_span)  
        val stressLevel = endocrineSystem.hormones.cortisol.value  
  
        // 2. Gradient-Bounded Memory Retrieval  
        // High stress (1.0) = limit to 1 memory (shallow reflex)  
        // Low stress (0.1) = retrieve up to 10 memories (deep analysis)  
        val maxMemories = max(1, (10 * (1.0f - stressLevel)).toInt())  
  
        // Retrieve context using the dynamically bounded limit
```

```

    [span_28](start_span)[span_29](start_span)val context =
memory.retrieveRelevantMemory(input.perception, limit = maxMemories)
//[span_28](end_span)[span_29](end_span)

    [span_30](start_span)[span_31](start_span)// 3. Modulated
Reasoning[span_30](end_span)[span_31](end_span)
    if (stressLevel > 0.8f) {
        [span_32](start_span)Decision("EMERGENCY_EVASION", confidence = 0.9f)
//[span_32](end_span)
    } else {
        [span_33](start_span)Decision("ANALYTICAL_RESPONSE", confidence = 0.7f)
//[span_33](end_span)
    }
}
}

```

Hebbian Associative Graphing

```

import java.util.concurrent.ConcurrentHashMap

[span_36](start_span)class AssociativeMemory { //[span_36](end_span)
    // Node -> (Connected Node -> Synaptic Weight)
    private val neuralGraph = ConcurrentHashMap<String, ConcurrentHashMap<String,
Float>>()

    // Long-Term Potentiation (Hebbian Learning)
    fun fireTogetherWireTogether(conceptA: String, conceptB: String, dopamineLevel: Float) {
        val edges = neuralGraph.getOrPut(conceptA) { ConcurrentHashMap() }
        val currentWeight = edges.getDefault(conceptB, 0.0f)

        [span_37](start_span)// Dopamine acts as a learning multiplier[span_37](end_span)
        val learningRate = 0.05f * (1.0f + dopamineLevel)
        edges[conceptB] = min(1.0f, currentWeight + learningRate)
    }

    [span_38](start_span)// Long-Term Depression (Sleep Cycle Pruning)[span_38](end_span)
    [span_39](start_span)fun sleepCycleDecay(decayFactor: Float = 0.02f) {
//[span_39](end_span)
        neuralGraph.forEach { (node, edges) ->
            edges.forEach { (connectedNode, weight) ->
                val newWeight = weight - decayFactor
                if (newWeight <= 0.0f) {
                    [span_40](start_span)edges.remove(connectedNode) // Prune weak
association[span_40](end_span)

```

```
    } else {  
        edges[connectedNode] = newWeight  
    }  
}  
}  
}
```